



誠朴雄偉
勵學敦行

复习课（一）

（词法解析与LL(1)文法）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn





第一题



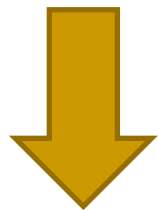
题目 1 (正则表达式与自动机 (15 = 7 + 8 分))

考虑正则表达式 $r = (a \mid b)^*a^+$ 。

- (1) 请使用 Thompson 构造法构造等价的 NFA。
- (2) 请使用子集构造法构造等价的 DFA。(注: 请标明状态之间的对应关系)



词素



词法单元

我们已有的工具:

- 1.RE
- 2.DFA
- 3.NFA



正则表达式到自动机



- 正则表达式可以简洁、精确地描述词法单元的模式
- 进行模式匹配时，模拟**DFA**的执行比模拟**NFA**更为简单有效
 - ϵ 的模拟，难
 - 不确定性的模拟，难
- 因此，需要将正则表达式转换为**DFA**
- 步骤：
 - 正则表达式到**NFA**
 - **NFA**到**DFA**



正则表达式到NFA



- 输入：字母表 Σ 上的一个正则表达式 r
- 输出：一个接受 $L(r)$ 的NFA N
- 基本思想
 - 根据正则表达式的递归定义，按照正则表达式的结构递归地构造出相应的NFA
 - 算法分成两个部分：
 - 基本规则处理 ϵ 和单符号的情况
 - 对于每个正则表达式的运算，建立构造相应NFA的方法

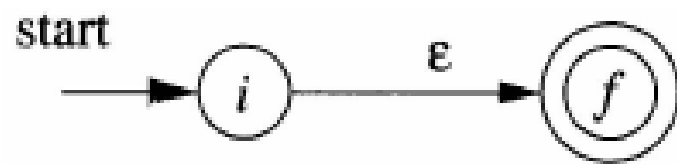


转换算法(1)

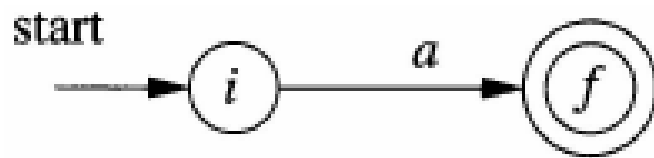


■ 基本规则:

○ 表达式 ϵ ,



○ 表达式 a ,





转换算法(2)



■ 归纳规则

- 正则表达式 s 和 t 的NFA分别是 $N(s)$ 和 $N(t)$
- $r=s|r$, r 的NFA $N(r)$

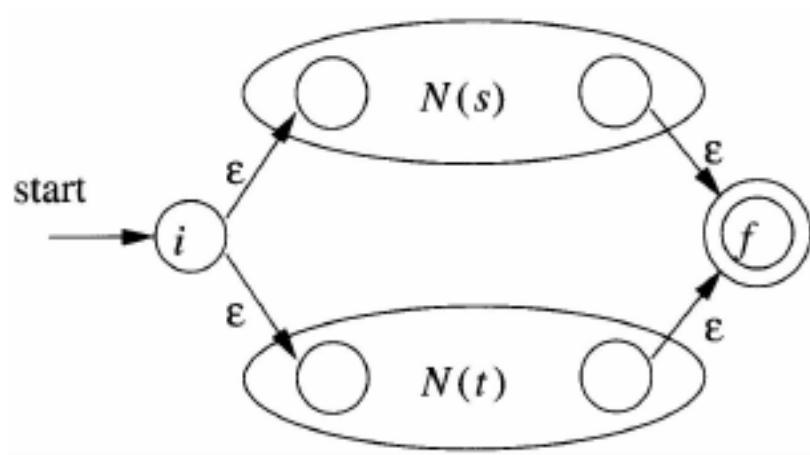


图 3-40 两个正则表达式的并的 NFA



转换算法(2)



- 归纳规则
 - 正则表达式 $r=st$, $N(r)$

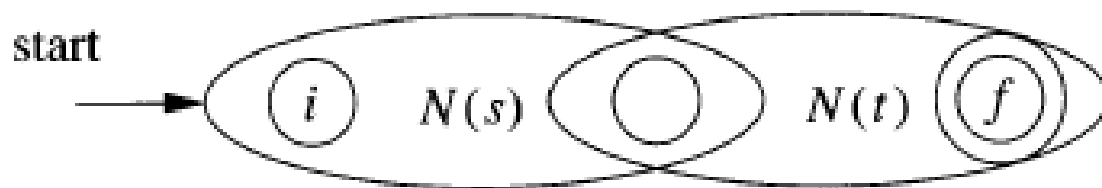


图 3-41 两个正则表达式的连接的 NFA



转换算法(3)



- 归纳规则
 - 正则表达式 $r=s^*$, $N(r)$

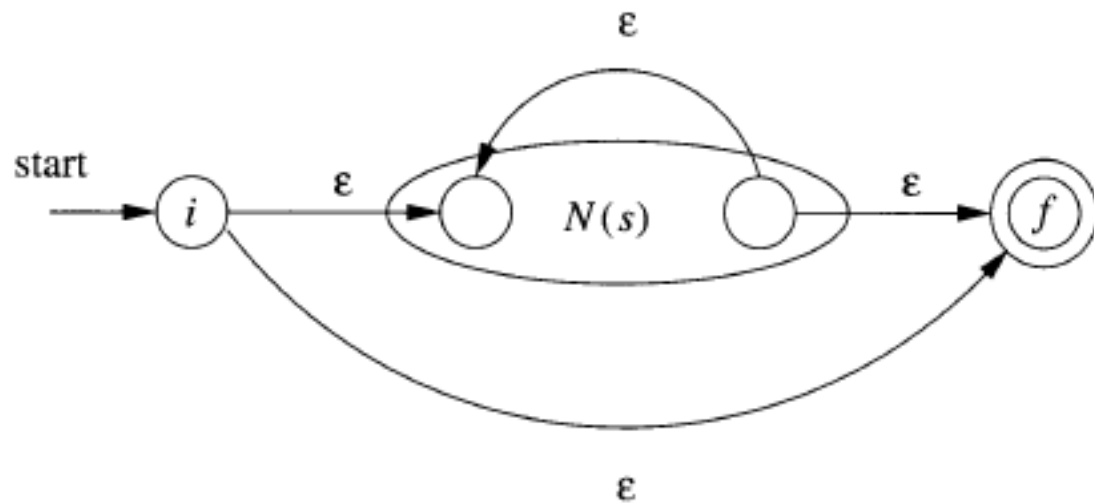


图 3-42 一个正则表达式的闭包的 NFA

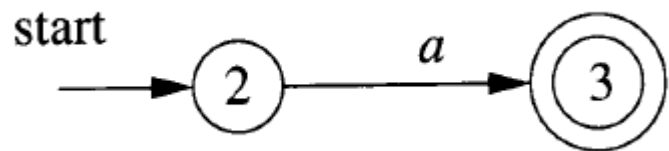
- $r=(s)$, $N(r)=N(s)$



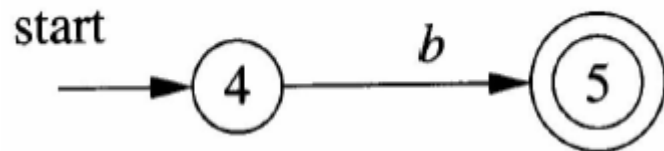
正则表达式到NFA的例子 (1)



- 正则表达式 $(a|b)^*abb$
- 第一个 a 对应的NFA



- 第一个 b 对应的NFA

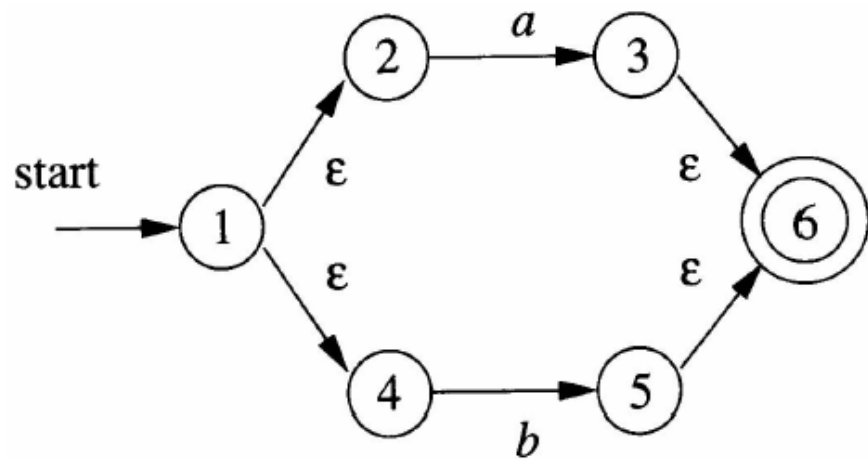




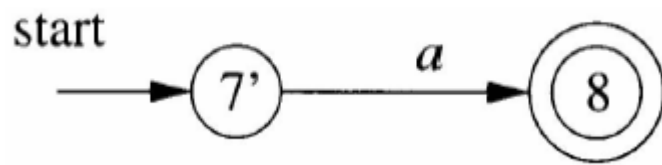
正则表达式到NFA的例子（2）



■ $(a|b)$ 的NFA



■ 第二个a的NFA





正则表达式到NFA的例子 (3)



■ $(a|b)^*$ 的NFA

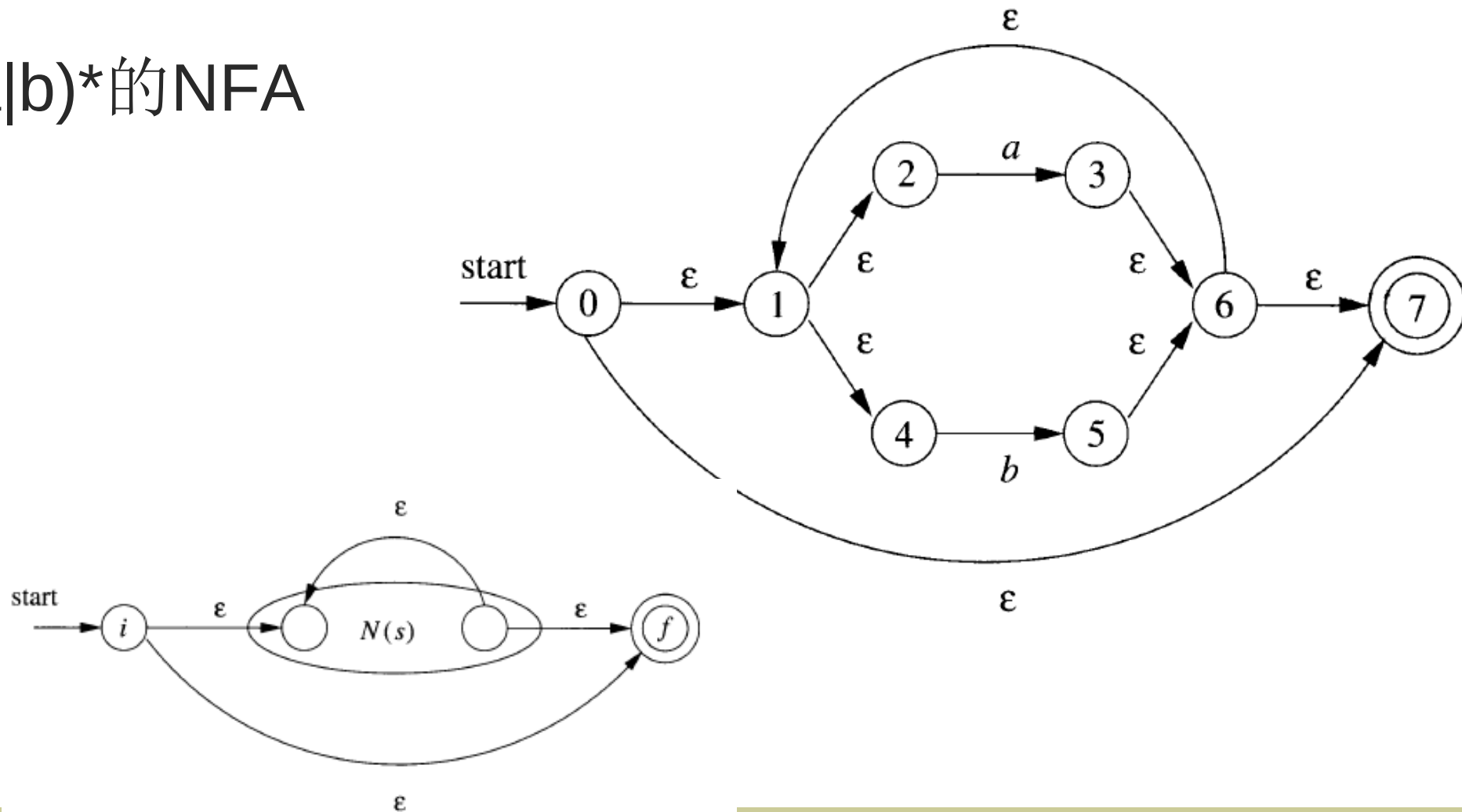


图 3-42 一个正则表达式的闭包的 NFA



NFA转换成DFA - 子集构造法



- 对NFA的模拟往往不如对DFA的模拟直接，除非转换花费更多的时间
- 基本思想： DFA每个状态 $\leftrightarrow$ NFA一个状态集
 - “并行地模拟” NFA在遇到一个给定输入串时可能执行的所有动作
 - 构造得到的DFA的每个状态和NFA的状态子集对应
 - DFA读入 $a_1, a_2, \dots, a_n$ 后到达的状态对应于从NFA开始状态出发沿着 $a_1, a_2, \dots, a_n$ 可能到达的状态集合
- 理论上，最坏情况下DFA的状态个数会是NFA状态个数的指数多个。但是对于大部分应用，NFA和相应的DFA的状态数量大致相同



NFA转换成DFA - 子集构造法



- 输入：一个NFA N
 - 输出：一个接受相同语言的DFA D
- s 表示 N 中的单个状态， T 代表 N 的一个状态集

操作	描述
$\epsilon\text{-closure}(s)$	能够从NFA的状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合
$\epsilon\text{-closure}(T)$	能够从 T 中某个NFA状态 s 开始只通过 ϵ 转换到达的NFA 状态集合，即 $\cup_{s \in T} \epsilon\text{-closure}(s)$.
$\text{move}(T, a)$	能够从 T 中某个状态 s 出发通过标号为 a 的转换到达的 NFA 状态的集合

图 3-31 NFA 状态集上的操作



NFA转换成DFA - 子集构造法



- D的开始状态是 ϵ -closure(s_0), D的接受状态是所有至少包含了N的一个接受状态的状态集合。

```
一开始,  $\epsilon$ -closure( $s_0$ )是  $Dstates$  中的唯一状态, 且它未加标记;  
while ( 在  $Dstates$  中有一个未标记状态  $T$  ) {  
    给  $T$  加上标记;  
    for ( 每个输入符号  $a$  ) {  
         $U = \epsilon$ -closure( $move(T, a)$ );  
        if (  $U$  不在  $Dstates$  中 )  
            将  $U$  加入到  $Dstates$  中, 且不加标记;  
         $Dtran[T, a] = U$ ;  
    }  
}
```

图 3-32 子集构造法



NFA转换成DFA - 子集构造法



- 对NFA的任何状态集合 ϵ -closure(T)的计算
- 一个图搜索过程

```
将 $T$ 的所有状态压入 $stack$ 中；  
将 $\epsilon$ -closure( $T$ )初始化为 $T$ ；  
while (  $stack$  非空 ) {  
    将栈顶元素 $t$ 弹出栈中；  
    for ( 每个满足如下条件的  $u$ : 从  $t$  出发有一个标号为 $\epsilon$ 的转换到达状态 $u$  )  
        if (  $u$  不在  $\epsilon$ -closure( $T$ ) 中 ) {  
            将 $u$ 加入到 $\epsilon$ -closure( $T$ )中；  
            将 $u$ 压入栈中；  
        }  
}
```

图 3-33 计算 ϵ -closure(T)



NFA到DFA转换的示例



- A: $=\epsilon\text{-closure}(0)=\{0,1,2,4,7\}$
- B: $\text{Dtran}[A,a]=\epsilon\text{-closure}(\text{move}(A,a))=\epsilon\text{-closure}(\{3,8\})=\{1,2,3,4,6,7,8\}$
- C: $\text{Dtran}[A,b]=\epsilon\text{-closure}(\text{move}(A,b))=\epsilon\text{-closure}(\{5\})=\{1,2,4,5,6,7\}$
- D: $\text{Dtran}[B,b]=\epsilon\text{-closure}(\text{move}(B,b))=\{1,2,4,5,6,7,9\}$
-
- E: $\text{Dtran}[D,b]=\epsilon\text{-closure}(\text{move}(B,b))=\{1,2,4,5,6,7,10\}$

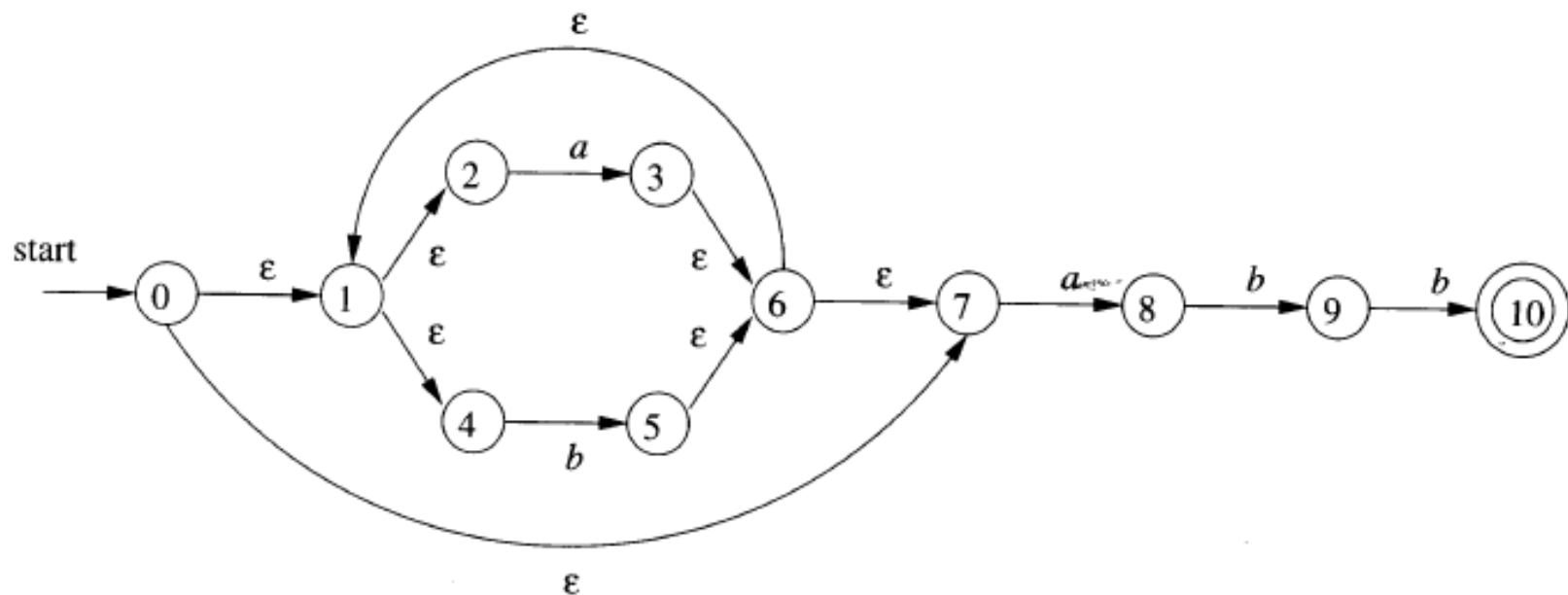


图 3-34 $(a|b)^*abb$ 对应的 NFA N



NFA到DFA转换的示例



- 开始状态: A
- 接受状态: E

NFA 状态	DFA 状态	a	b
{0, 1, 2, 4, 7}	A	B	C
{1, 2, 3, 4, 6, 7, 8}	B	B	D
{1, 2, 4, 5, 6, 7}	C	B	C
{1, 2, 4, 5, 6, 7, 9}	D	B	E
{1, 2, 4, 5, 6, 7, 10}	E	B	C

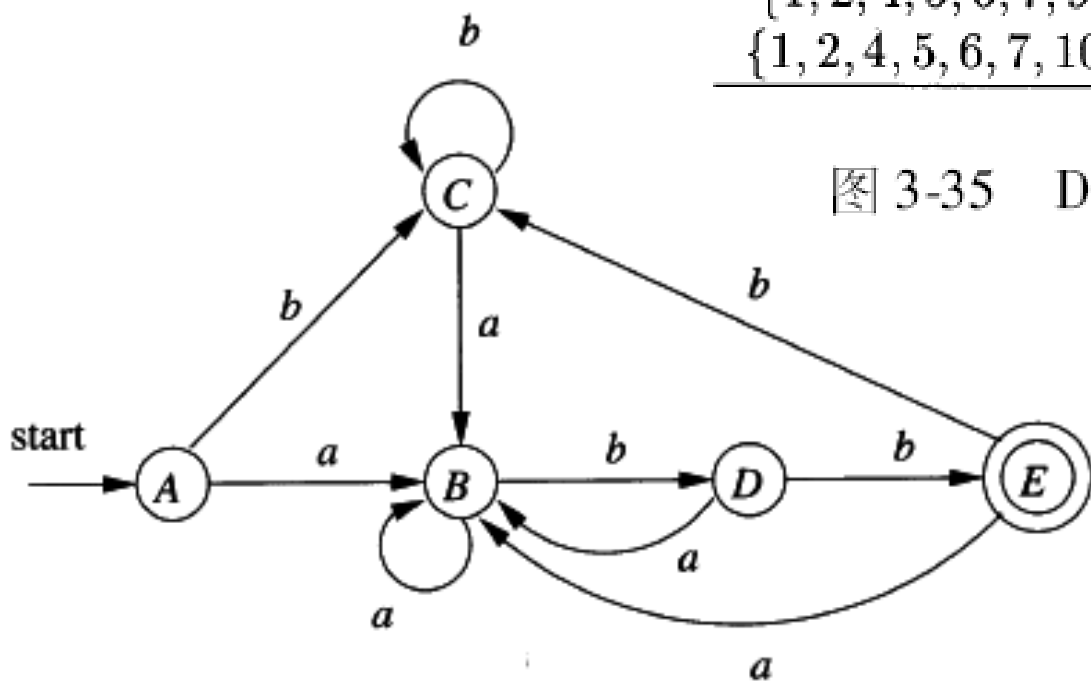


图 3-35 DFA D 的转换表 $Dtran$



对NFA的运行进行模拟:子集构造法



- 输入：一个以文件结束符`eof`结尾的输入串 x ，一个NFA N ，其开始状态是 S_0 ，接受状态集为 F ，转换函数为 $move$
- 输出：如果 N 接受 x ，返回`yes`，否则返回`no`

```
1)  $S = \epsilon\text{-closure}(s_0);$   
2)  $c = nextChar();$   
3) while (  $c \neq eof$  ) {  
4)      $S = \epsilon\text{-closure}(move(S, c));$   
5)      $c = nextChar();$   
6) }  
7) if (  $S \cap F \neq \emptyset$  ) return "yes";  
8) else return "no";
```

图 3-37 模拟的一个 NFA



第一题



题目 1 (正则表达式与自动机 (15 = 7 + 8 分))

考虑正则表达式 $r = (a \mid b)^*a^+$ 。

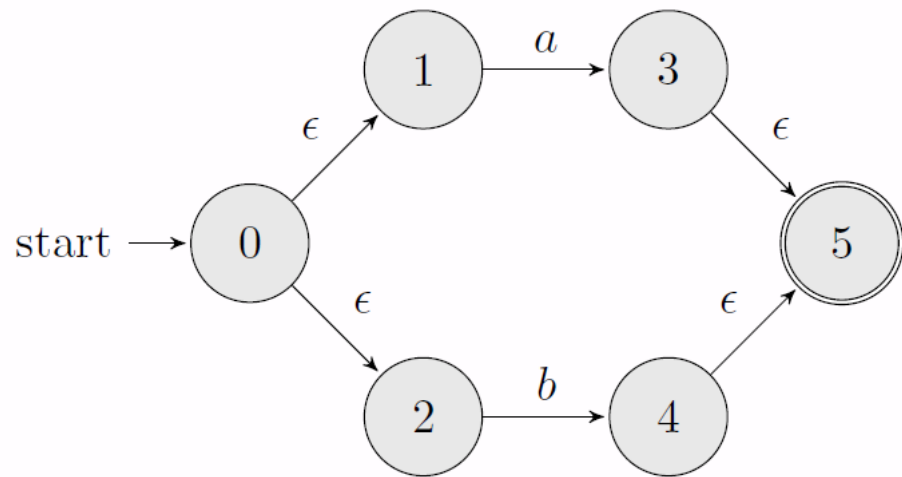
- (1) 请使用 Thompson 构造法构造等价的 NFA。
- (2) 请使用子集构造法构造等价的 DFA。(注: 请标明状态之间的对应关系)



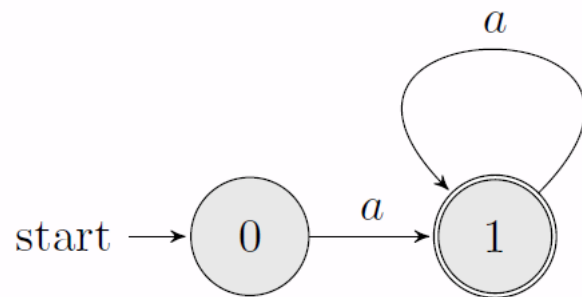
第一题



$a \mid b$:



a^+ :

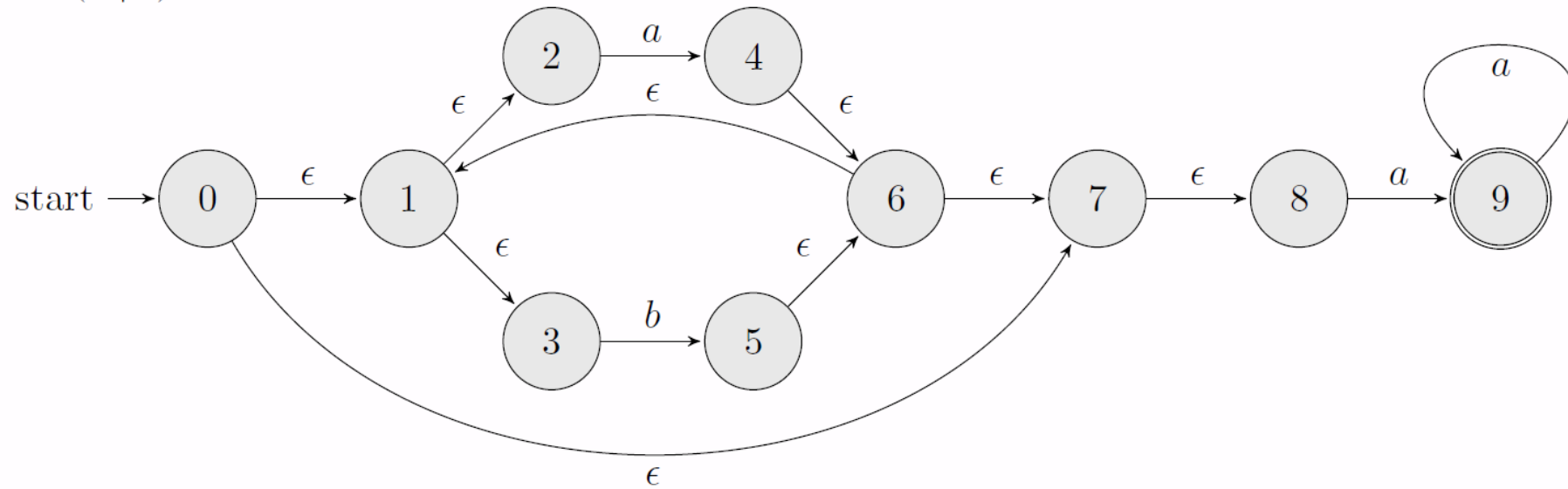




第一题



$r = (a \mid b)^* a^+$:





第一题



$$A = \epsilon\text{-CLOSURE}(\{0\}) = \{0, 1, 2, 3, 7, 8\}$$

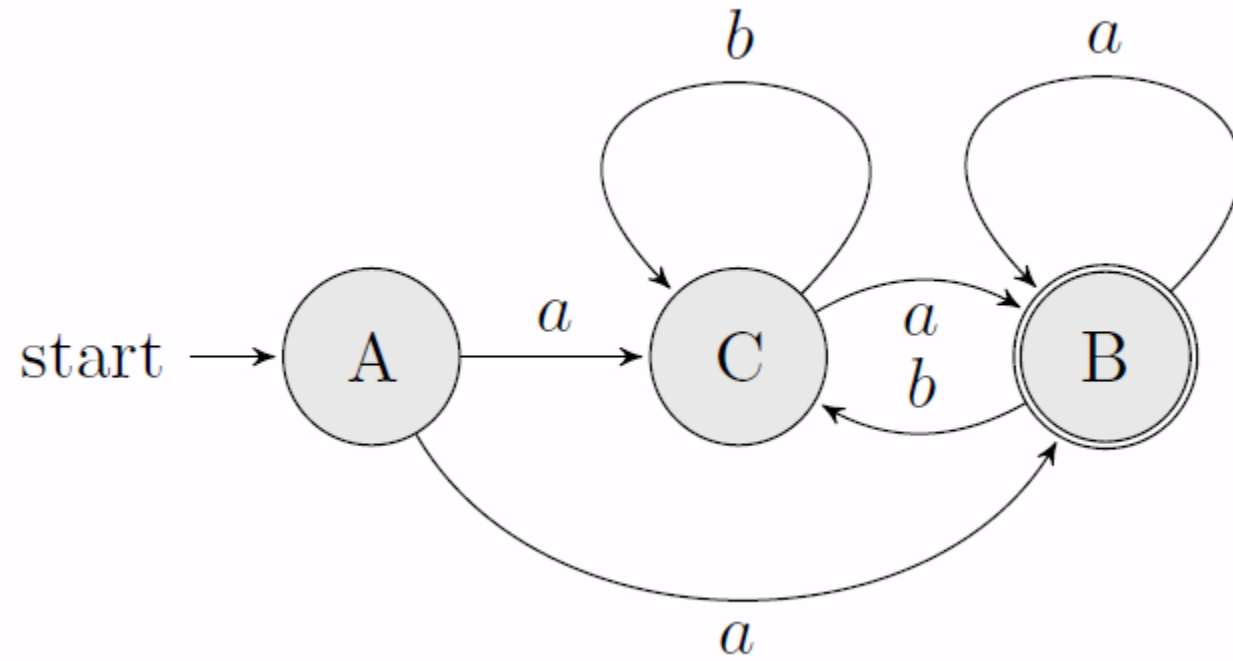
$$B = \epsilon\text{-CLOSURE}(\text{MOVE}(A, a)) = \epsilon\text{-CLOSURE}(\{4, 9\}) = \{1, 2, 3, 4, 6, 7, 8, 9\}$$

$$C = \epsilon\text{-CLOSURE}(\text{MOVE}(A, b)) = \epsilon\text{-CLOSURE}(\{5\}) = \{1, 2, 3, 5, 6, 7, 8\}$$

NFA	DFA	a	b
$\{0, 1, 2, 3, 7, 8\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8, 9\}$	B	B	C
$\{1, 2, 3, 5, 6, 7, 8\}$	C	B	C



第一题





第二题



考虑一个密码强度验证系统，需要识别符合以下规则的密码字符串：

- 标识符由字母 **a**、**b** 和数字 **0**、**1** 组成
- 标识符必须以字母开头
- 标识符必须以 **abb** 结尾
- 标识符中间部分不能包含连续的两个数字（即不能出现 **00**、**01**、**10**、**11**）

例如：

合法标识符：**aabb**, **babb**, **ababb**, **a0aabb**, **b1babb**

非法标识符：**0abb**（以数字开头），**ab**（不以**abb**结尾），**a00abb**（包含连续数字），**a01abb**（包含连续数字）

请完成以下任务：

- 写出描述这类字符串的正则表达式
- 使用Thompson构造法构造与该正则表达式等价的非确定有限自动机（NFA）
- 使用子集构造法将NFA转换为确定有限自动机（DFA）
- 最小化得到的DFA



第二题



考虑一个密码强度验证系统，需要识别符合以下规则的密码字符串：

- 标识符由字母 **a**、**b** 和数字 **0**、**1** 组成
- 标识符必须以字母开头
- 标识符必须以 **abb** 结尾
- 标识符中间部分不能包含连续的两个数字（即不能出现 **00**、**01**、**10**、**11**）

例如：

合法标识符：**aabb**, **babb**, **ababb**, **a0aabb**, **b1babb**

非法标识符：**0abb**（以数字开头），**ab**（不以**abb**结尾），**a00abb**（包含连续数字），**a01abb**（包含连续数字）

1. 以字母开头： $(a|b)$
 2. 以 **abb** 结尾：**abb**
 3. 中间部分不能包含连续数字：中间可以是字母，或者数字后必须跟字母
- 正则表达式为：

$$(a|b)((a|b)|0(a|b)|1(a|b))*abb$$



第三题



题目 2 (LL 语法分析 (20 = 8 + 8 + 4 分))

考虑文法 G :

$$S \rightarrow cS \quad (1)$$

$$S \rightarrow AB \quad (2)$$

$$A \rightarrow aA \quad (3)$$

$$A \rightarrow \epsilon \quad (4)$$

$$B \rightarrow bBa \quad (5)$$

$$B \rightarrow d \quad (6)$$

(1) 请为文法 G 计算必要的 FIRST 集合与 FOLLOW 集合;

(2) 请为文法 G 构造预测分析表;

(3) 请问文法 G 是否是 $LL(1)$ 文法, 并说明理由。



消除左递归



- 左递归的定义
 - 文法中一个非终结符号**A**使得对某个串 α , 存在一个推导 $A \xRightarrow{+} A\alpha$ 则称这个文法是左递归的
- 立即左递归
 - 如果存在 $A \rightarrow A\alpha$, 则称为立即左递归
- 为什么要消除左递归?
 - 自顶向下的语法分析技术不能处理左递归的文法
- 立即左递归的消除实例:

$$\boxed{A \rightarrow A\alpha \mid \beta} \Longrightarrow \boxed{\begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \varepsilon \end{array}}$$



立即左递归的消除



- 假设非终结符号A存在立即左递归的情形，假设以A为左部的规则有：

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

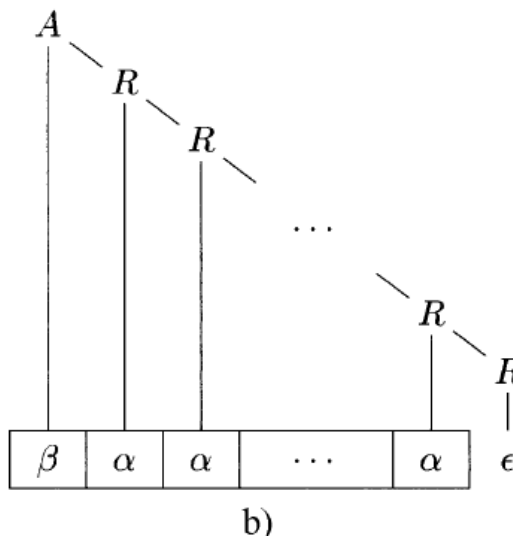
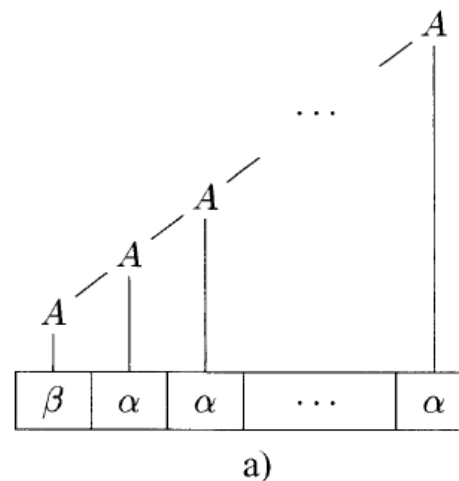
首先对A产生式**分组** (所有 α_i 不等于 ϵ , β_i 不以A开头)

可以**替换**为

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon$$

- 由A生成的串总是以某个 β_i 开头，然后跟上零个或者多个 α_i 的重复





消除立即左递归



- $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

- 分组

$$A \rightarrow \boxed{A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m} \mid \boxed{\beta_1 \mid \beta_2 \mid \dots \mid \beta_n}$$

- 替换

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$



立即左递归消除示例



$$\begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid \mathbf{id} \end{array} \quad \Longrightarrow \quad \begin{array}{l} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \epsilon \\ T \rightarrow FT' \\ T' \rightarrow * FT' \mid \epsilon \\ F \rightarrow (E) \mid \mathbf{id} \end{array}$$



消除多步左递归



- 消除立即左递归的方法并不能消除因为两步或多步推导而产生的左递归
- 文法: $S \rightarrow Aa \mid b, A \rightarrow Ac \mid Sd \mid \varepsilon$
- $S \Rightarrow Aa \Rightarrow Sda$
- 如何消除?



消除算法



- 输入：没有环 A_i A_i 和 $\overset{+}{\Rightarrow} \rightarrow \varepsilon$
- 输出：一个等价的无左递归的文法
- 算法原理：
 - 给非终结符号**排序**， $A_1, A_2, \dots, A_n$
 - 如果只有 $A_i \Rightarrow A_j$ ($i < j$)，则不会有左递归
 - 如果发现 $A_i \Rightarrow A_j$ ($i > j$)，**代入** A_j 的当前产生式，若替换后有 A_i 的直接左递归，再**消除**



通用的左递归消除方法



- 输入：没有环和 ϵ 产生式的文法G
- 输出：等价的无左递归的文法
- 步骤：
将文法的非终结符号任意排序为 $A_1, A_2, \dots, A_n$
for i=1 to n do {
 for j = 1 to i-1 do
 {
 将形如 $A_i \rightarrow A_j \gamma$ 的产生式替换为 $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$,
 其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ 是以 A_j 为左部的所有产生式;
 }
 消除 A_i 的立即左递归;
}



通用的左递归消除方法（续）



1. 内层循环的每一次迭代保证了 A_i 的产生式的右部首符号都比 A_j 更加靠后
2. 当内层循环结束时， A_i 的产生式的右部首符号不先于 A_i
3. 消除立即左递归就保证了 A_i 的产生式的右部首符号必然比 A_i 后。
（不能有环和 ϵ 产生式）
4. 当外层循环结束时，所有的产生式都是如此。使用这些产生式当然不会产生左递归的情形



消除算法（示例）



- $S \rightarrow Aa \mid b, \quad A \rightarrow Ac \mid Sd \mid \varepsilon$
- 排序: $S \ A$
- 替换:
 - $i=1$, 没有处理
 - $i=2$, 替换 $A \rightarrow Sd$ 中的 S , 得到 $A \rightarrow Ac \mid Aad \mid bd \mid \varepsilon$
- 消除立即左递归

$$S \rightarrow A a \mid b$$

$$A \rightarrow b d A' \mid A'$$

$$A' \rightarrow c A' \mid a d A' \mid \epsilon$$



自顶向下分析技术



- 自顶向下分析可以被看作是为输入串构造语法分析树的问题，也可以看作一个寻找输入串的**最左推导**的过程
- 问题
 - 在推导的每一步，对非终结符号 A ，应用哪个产生式，以可能产生于输入串相匹配的终结符号串



id+id*id的自顶向下分析



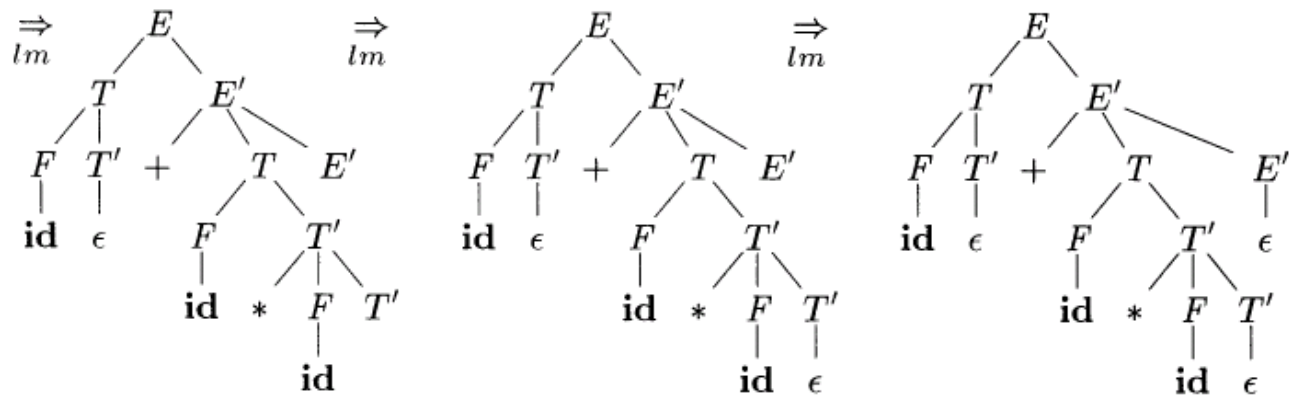
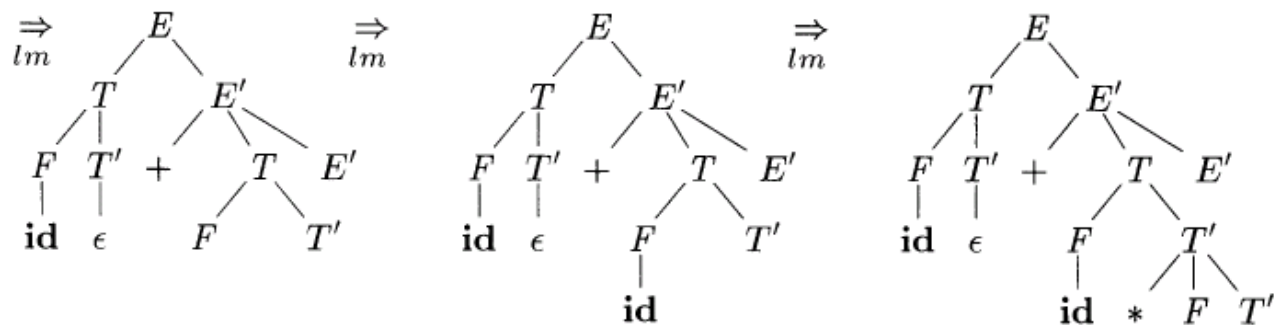
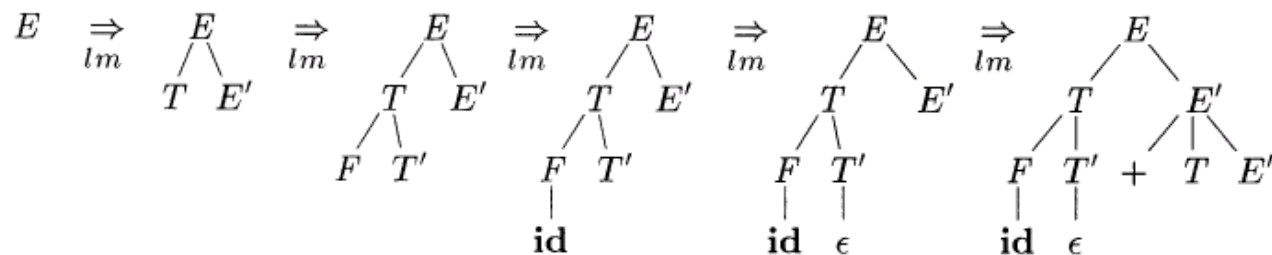
$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$F \rightarrow (E) \mid \mathbf{id}$$





自顶向下语法分析



- 问题：对于非终结符号A，选择哪一个产生式
- 一种通用的递归下降分析框架
 - 由一组过程组成，每个非终结符号对应一个过程
 - 程序的执行从开始符号对应的过程开始
 - 每个过程的功能是：选择一个产生式体，扫描相应的句子。若遇到非终结符号，调用该符号对应的过程

```
void A() {  
  1)  选择一个 A 产生式,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
  2)  for (  $i = 1$  to  $k$  ) {  
  3)      if (  $X_i$  是一个非终结符号 )  
  4)          调用过程  $X_i()$ ;  
  5)      else if (  $X_i$  等于当前的输入符号  $a$  )  
  6)          读入下一个输入符号;  
  7)      else /* 发生了一个错误 */;  
  }  
}
```



递归下降分析过程示例



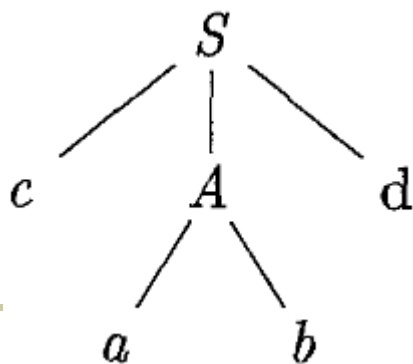
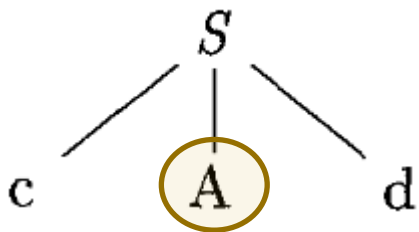
■ 输入串 $w = c a d$

■ 文法:

○ $S \rightarrow cAd$

○ $A \rightarrow ab \mid a$

```
void A() {  
    1) 选择一个  $A$  产生式,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
    2)  for (  $i = 1$  to  $k$  ) {  
    3)      if (  $X_i$  是一个非终结符号 )  
    4)          调用过程  $X_i()$ ;  
    5)      else if (  $X_i$  等于当前的输入符号  $a$  )  
    6)          读入下一个输入符号;  
    7)      else /* 发生了一个错误 */;  
    }  
}
```





递归下降分析过程示例



■ 输入串 $w = c a d$

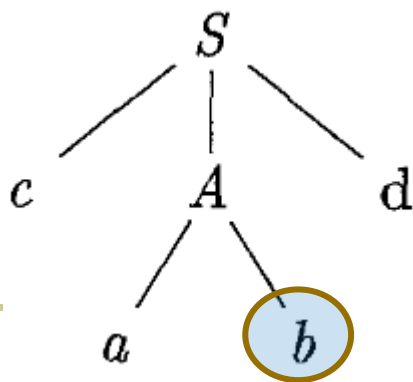
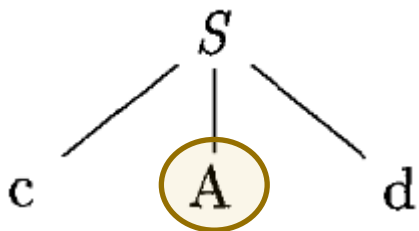


■ 文法:

○ $S \rightarrow cAd$

○ $A \rightarrow ab \mid a$

```
void A() {  
    1) 选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
    2)  for (  $i = 1$  to  $k$  ) {  
    3)      if (  $X_i$  是一个非终结符号 )  
    4)          调用过程  $X_i()$ ;  
    5)      else if (  $X_i$  等于当前的输入符号  $a$  )  
    6)          读入下一个输入符号;  
    7)      else /* 发生了一个错误 */;  
    }  
}
```





递归下降分析过程示例

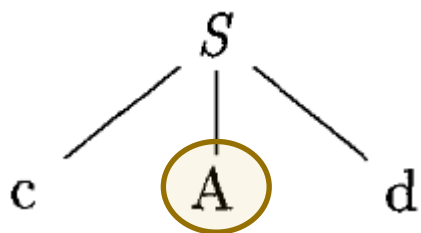


■ 输入串 $w = c a d$

■ 文法:

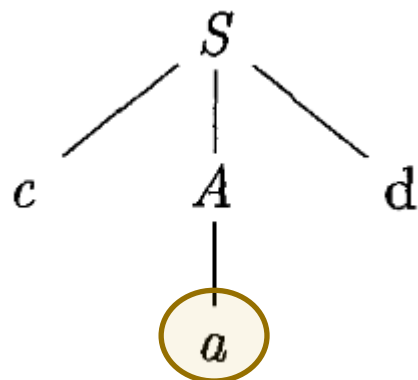
○ $S \rightarrow cAd$

○ $A \rightarrow ab \mid a$



1) 2) 3) 4) 5) 6) 7)

```
void A() {  
    1) 选择一个  $A$  产生式,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
    2)  for (  $i = 1$  to  $k$  ) {  
    3)      if (  $X_i$  是一个非终结符号 )  
    4)          调用过程  $X_i()$ ;  
    5)      else if (  $X_i$  等于当前的输入符号  $a$  )  
    6)          读入下一个输入符号;  
    7)      else /* 发生了一个错误 */;  
    }  
}
```





递归下降分析过程示例



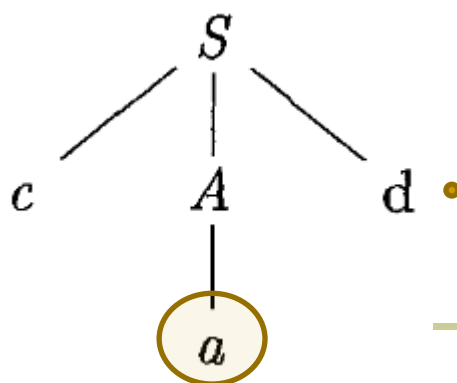
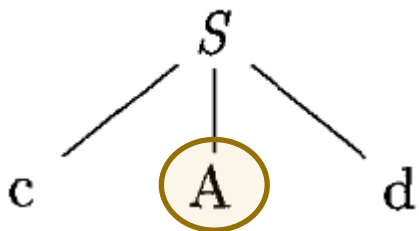
■ 输入串 $w = c a d$

■ 文法:

○ $S \rightarrow cAd$

○ $A \rightarrow ab \mid a$

```
void A() {  
    1) 选择一个  $A$  产生式,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
    2)  for (  $i = 1$  to  $k$  ) {  
    3)      if (  $X_i$  是一个非终结符号 )  
    4)          调用过程  $X_i()$ ;  
    5)      else if (  $X_i$  等于当前的输入符号  $a$  )  
    6)          读入下一个输入符号;  
    7)      else /* 发生了一个错误 */;  
    }  
}
```



匹配成功!



递归下降过程示例(续)



- 可能需要回溯，或者说，可能需要重复扫描输入
- “在回到A时，我们必须把输入指针重新设置到位置2，即我们第一次尝试展开A时该指针指向的位置。这意味着A的过程必须将输入指针存放在一个局部变量中。”
- 如果文法中存在左递归.....
- 回溯（穷试 ☺）显然是笨办法



预测分析法简介



- 试图从开始符号推导出输入符号串
- 以开始符号作为初始的当前句型
- 每次为最左边的非终结符号选择适当的产生式
 - 通过查看下一个输入符号来选择这个产生式
 - 有多个可能的产生式时预测分析法无能为力
- 比如文法: $E \rightarrow +EE \mid -EE \mid id$; 输入为+ **id** - **id** **id**
- 当两个产生式具有相同的前缀时无法预测
 - 文法: $stmt \rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \mid \text{if } expr \text{ then } stmt$
 - 输入: **if** a **then** ...
 - 新文法: $stmt \rightarrow \text{if } expr \text{ then } stmt \text{ elsePart}$
 - $elsePart \rightarrow \text{else } stmt \mid \epsilon$
- 需要提取公因子



预测分析技术

消除二义性
消除左递归
提取左公因子



- 一种确定性的、无回溯的分析技术
- 在每一步选择正确的产生式

□ 例1: 文法G(S):

$S \rightarrow pA$

$S \rightarrow qB$

$A \rightarrow cAd$

$A \rightarrow a$

□ 输入串 $W=pccadd$



预测分析技术（续）



□ 例2：文法**G(S)**为：

$$S \rightarrow Ap$$

$$S \rightarrow Bq$$

$$A \rightarrow a \mid cA$$

$$B \rightarrow b \mid dB$$

□ 输入**W=ccap**



预测分析技术（续）



- 通过在输入中向前看固定多个符号来选择正确的产生式
- 通常情况下，我们只需要向前看一个符号
 - 递归下降分析一般只适合于每个子表达式的第一个终结符能够为产生式选择提供足够信息的那些文法
- 给出两个与文法相关的两个函数
 - FIRST
 - FOLLOW
- 基于上述两个函数，可以根据下一个输入符号来选择应用哪个产生式



FIRST和FOLLOW (1)

很重要



- 在自顶向下的分析技术中，通常使用向前看几个符号来唯一地确定产生式（这里假定只看一个符号）
- 当前句型是 $xA\beta$ ，而输入是 $xa\dots$ 。那么选择产生式 $A \rightarrow \alpha$ 的必要条件是下列之一：
 - $A \xRightarrow{*} \epsilon$ ，且 β 以 a 开头；也就是说在某个句型中 a 跟在 A 之后；
 - $A \xRightarrow{*} a\dots$ ；
 - 如果按照这两个条件选择时能够保证唯一性，那么我们就可以避免回溯。
- 因此，我们定义FIRST和FOLLOW



FIRST和FOLLOW (2)



■ FIRST(α):

- 可以从 α 推导得到的串的首符号的集合;
- 如果 $\alpha \xRightarrow{*} \varepsilon$, 那么 ε 也在 FIRST(α)中;

■ FOLLOW(A):

- 可能在某些句型中紧跟在A右边的终结符号的集合。

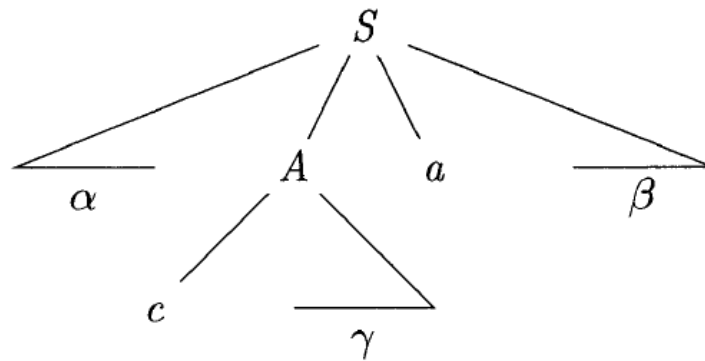


图 4-15 终结符号 c 在 FIRST(A)
中且 a 在 FOLLOW(A) 中



FIRST (α)



- 定义：可从 α 推导得到的串的首符号的集合，其中 α 是任意的文法符号串。如果 $\alpha \xRightarrow{*} \varepsilon$ ，那么 ε 也在FIRST(α)中
- FIRST函数的意义
 - 如果两个A产生式 $A \rightarrow \alpha \mid \beta$ ，其中First(α)和First(β)是不相交的集合。下一个输入符号是a，若 $a \in \text{First}(\alpha)$ ，则选择 $A \rightarrow \alpha$ ，若 $a \in \text{First}(\beta)$ ，则选择 $A \rightarrow \beta$
 - 用于预测产生式的选择



First的计算



- 对于文法符号 X 的 $FIRST(X)$ ，通过不断应用下列规则，直到没有新的终结符号或者 ϵ 可以加入到任何 $FIRST$ 集合为止
 - 如果 X 是终结符号，那么 $FIRST(X)=\{X\}$
 - 如果 X 是非终结符号，且有规则 $X \rightarrow a...$ ，那么将 a 添加到 $FIRST(X)$ 中；如果 $X \rightarrow \epsilon$ ，那么 ϵ 也在 $FIRST(X)$ 中
 - 对于规则 $X \rightarrow Y_1Y_2...Y_n$ ，把 $FIRST(Y_1)$ 中的非 ϵ 符号添加到 $FIRST(X)$ 中。如果 ϵ 在 $FIRST(Y_1)$ 中，把 $FIRST(Y_2)$ 中的非 ϵ 符号添加到 $FIRST(X)$ 中...；如果 ϵ 在 $FIRST(Y_n)$ 中，把 ϵ 添加到 $FIRST(X)$ 中



First的计算（续）



- 对于文法符号串 $X_1X_2...X_n$ 的First集合
 - 向 $\text{First}(X_1X_2...X_n)$ 加入 $\text{First}(X_1)$ 中所有的非 ϵ 符号。
 - 如果 ϵ 在 $\text{First}(X_1)$ 中，再加入 $\text{First}(X_2)$ 中的所有非 ϵ 符号。如果 ϵ 在 $\text{First}(X_1)$ 和 $\text{First}(X_2)$ 中，再加入 $\text{First}(X_3)$ 中的所有非 ϵ 符号。依次类推。
 - 如果对所有的 i （1到 n ）， ϵ 都在 $\text{First}(X_i)$ 中，则 ϵ 加入 $\text{First}(X_1X_2...X_n)$ 中。



First的计算示例



$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$F \rightarrow (E) \mid \mathbf{id}$$

- $\text{First}(F) = \text{First}(T) = \text{First}(E) = \{ (, \text{id} \}$
- $\text{First}(E') = \{ +, \epsilon \}$
- $\text{First}(T') = \{ *, \epsilon \}$
- $\text{First}(TE') = \text{First}(T) = \{ (, \text{id} \}$
- $\text{First}(+TE') = \{ + \}$
-



一个实例



■ 文法G[S], 输入bcd

- $S \rightarrow AB \mid CD$
- $A \rightarrow aD \mid \varepsilon$
- $C \rightarrow cD$
- $B \rightarrow bC$
- $D \rightarrow d$

- $\text{First}(S) = \{a, b, c\}$
- $\text{First}(A) = \{a, \varepsilon\}$
- $\text{First}(B) = \{b\}$
- $\text{First}(C) = \{c\}$
- $\text{First}(D) = \{d\}$



FOLLOW函数

文法G[S], 输入bcd

$S \rightarrow AB|CD$

$A \rightarrow aD| \epsilon$

$C \rightarrow cD$

$B \rightarrow bC$

$D \rightarrow d$



- 对于非终结符号A, FOLLOW(A)定义为可能在某些句型中紧跟在A右边的终结符号的集合
 - 例如 $S \xRightarrow{*} \alpha A a \beta$, 终结符号 $a \in \text{Follow}(A)$
- 如果A是某些句型的最右符号, 那么 $\$ \in \text{Follow}(A)$ 。\$是特殊的输入串“结束标记”
- FOLLOW函数的意义:
 - 如果 $A \rightarrow \alpha$, 当 $\alpha \rightarrow \epsilon$ 或 $\alpha \xRightarrow{*} \epsilon$ 时, FOLLOW(A)可以帮助我们做出选择恰当的产生式
 - 例如: 如果 $A \rightarrow \alpha$, b属于FOLLOW(A), 如果 $\alpha \xRightarrow{*} \epsilon$, 则若当前输入符号是b, 可以选择 $A \rightarrow \alpha$, 因为A最终到达了 ϵ , 而且后面跟着b



FOLLOW计算



- 计算各个非终结符号 A 的 $FOLLOW(A)$ 集合，不断应用下列规则，直到没有新的终结符号可以被加入到任意 $FOLLOW$ 集合中
 - 将 $\$$ 放入 $FOLLOW(S)$ ， S 是开始符号，而 $\$$ 是输入串的结束标记
 - 如果存在产生式 $A \rightarrow \alpha B \beta$ ，那么 $First(\beta)$ 中除 ϵ 之外的所有符号都在 $Follow(B)$ 中
 - 如果存在一个产生式 $A \rightarrow \alpha B$ ，或存在产生式 $A \rightarrow \alpha B \beta$ 且 $First(\beta)$ 包含 ϵ ，那么 $Follow(A)$ 中的所有符号都在 $Follow(B)$ 中



Follow的计算示例



□ 文法

$$E \rightarrow T E'$$

$$E' \rightarrow +T E' \mid \varepsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow *F T' \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

□ FOLLOW(E)={ } , \$}

$$\leftarrow \text{FIRST}() \cup \{\$ \}$$

□ FOLLOW(E')={ } , \$}

$$\leftarrow \text{FOLLOW}(E)$$

□ FOLLOW(T)={+,), \$}

$$\leftarrow \text{FIRST}(E') \cup \text{FOLLOW}(E')$$

□ FOLLOW(T')={+,), \$}

$$\leftarrow \text{FOLLOW}(T)$$

□ FOLLOW(F)={*, +,), \$}

$$\leftarrow \text{FIRST}(T') \cup \text{FOLLOW}(T)$$

- 将\$放入FOLLOW(S), S是开始符号, 而\$是输入串的结束标记
- 如果存在产生式 $A \rightarrow \alpha B \beta$, 那么First(β)中除 ε 之外的所有符号都在Follow(B)中
- 如果存在一个产生式 $A \rightarrow \alpha B$, 或存在产生式 $A \rightarrow \alpha B \beta$ 且First(β)包含 ε , 那么Follow(A)中的所有符号都在Follow(B)中



课堂练习



- 有如下文法:
 - $E \rightarrow \text{id } X$
 - $X \rightarrow \varepsilon \mid (A) \mid [E]$
 - $A \rightarrow EY$
 - $Y \rightarrow \varepsilon \mid ;A$
- 给出各个非终结符号的First集和Follow集合。



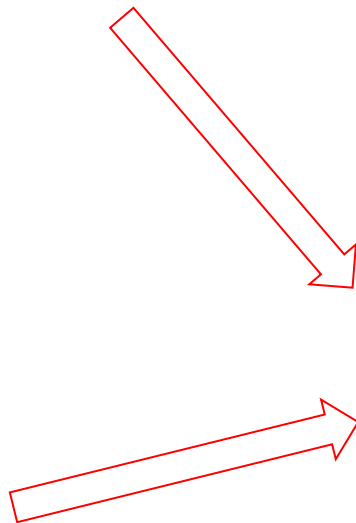
思考：为什么



- 消除二义性
- 消除左递归
- 提取左公因子
- 求First和Follow集合

改造文法

LL(1)文法
的预测分析





LL(1) 文法 (1)



- LL(1)文法，第一个L表示自左向右扫描，第二个L指产生最左推导，而1则表示向前看一个输入符号
- LL(1)文法可以利用不回溯的确定性的预测分析技术，因为只需要检查当前输入符号就可以为一个非终结符号选择正确的产生式



LL(1) 文法 (2)



- 定义:对于文法的任意两个不同的产生式 $A \rightarrow \alpha | \beta$
 - 不存在终结符号 a 使得 α 和 β 都可以推导出以 a 开头的串
 - α 和 β 最多只有一个可以推导出空串
 - 如果 β 可以推导出空串, 那么 α 不能推导出以FOLLOW 中任何终结符号开头的串;
- 等价于:
 - $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \Phi$; (条件一、二)
 - 如果 $\varepsilon \in \text{FIRST}(\beta)$, 那么 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \Phi$; 反之亦然。
(条件三)



LL(1) 文法 (3)



- 思考：对于一个LL(1)文法，我们期望从A推导出以终结符号a开头的符号串
 - 我们如何选择产生式？
 - 有多个选择吗？



LL(1) 文法 (4)



- 对于LL(1)文法, 可以在自顶向下分析过程中, 根据当前输入符号来确定使用的产生式
- 例如:
 - 产生式: $stmt \rightarrow \mathbf{if (exp) stmt \text{ else } stmt} \mid \mathbf{while (exp) stmt} \mid \mathbf{a}$
 - 输入: $\mathbf{if (exp) while (exp) a \text{ else } a}$
 - 首先选择产生式 $stmt \rightarrow \mathbf{if (exp) stmt \text{ else } stmt}$ 得到句型 $\mathbf{if (exp) stmt \text{ else } stmt}$
 - 然后发现要把句型中的第一个 $stmt$ 展开, 选择产生式 $stmt \rightarrow \mathbf{while (exp) stmt}$, 得到句型 $\mathbf{if (exp) while (exp) stmt \text{ else } stmt}$ 。
 - 然后再展开第一个 $stmt$, 得到 $\mathbf{if (exp) while (exp) a \text{ else } stmt}$
 - ...



LL(1) 文法的预测分析技术



- LL(1)文法 == > 预测分析表
- 将First和Follow集合中的信息放入一个预测分析表M[A,a]，该预测表告诉我们当非终结符号为A，当前输入符号为a时，要选择哪条产生式
- 预测分析表的构造
 - 若当前输入符号a在First(α)中，选择产生式 $A \rightarrow \alpha$
 - 若 $\alpha \xRightarrow{*} \epsilon$ ，如果a在Follow(A)中，选择产生式 $A \rightarrow \alpha$
 - 如果当前符号a是\$，且\$在Follow(A)中，则选择产生式 $A \rightarrow \alpha$

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		



预测分析表构造



- 输入：文法 G
- 输出：预测分析表 M
- 方法：对于文法 G 的每个产生式 $A \rightarrow \alpha$ ，进行如下处理
 - 对于 $\text{First}(\alpha)$ 中的每个终结符号 a ，将 $A \rightarrow \alpha$ 加入到 $M[A, a]$
 - 如果 ϵ 在 $\text{First}(\alpha)$ 中，那么对于 $\text{Follow}(A)$ 中的每个终结符号 b ，将 $A \rightarrow \alpha$ 加入到 $M[A, b]$ 中
 - 如果 ϵ 在 $\text{First}(\alpha)$ 中，且 $\$$ 在 $\text{Follow}(A)$ 中，将 $A \rightarrow \alpha$ 加入到 $M[A, \$]$ 中
- 完成上述操作后，若 $M[A, a]$ 中没有产生式，填为Error



预测分析表的例子

■ 文法:

- $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid \text{id}$

这个例子恰巧使得每个产生式的右部的第一个符号的FIRST集合就等于产生式右部的FIRST集合。但是在一般情况下不总是这样的。

■ FIRST集:

- $F: \{ (, \text{id} \};$ $E, T: \{ (, \text{id} \};$ $E': \{ +, \epsilon \};$ $T': \{ *, \epsilon \}$

■ FOLLOW集:

- $E: \{ \$,) \};$ $E': \{ \$,) \};$ $T, T': \{ +,), \$ \};$ $F: \{ +, *,), \$ \}$

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		



预测分析方法



- 对于任何文法**G**，都可以构造预测分析表
- 对于**LL(1)**文法，预测表中每个条目都唯一地指定了一个产生式，或者标明**Error**



二义性文法的预测分析表



- 对于某些文法，表中可能会有有一些多重定义的条目，比如左递归或二义性文法。

$S \rightarrow iEtSS' \mid a$ -FIRST(eS)={ e }, 使得 $S' \rightarrow eS$ 在 $M[S',e]$ 中;
 $S' \rightarrow eS \mid \epsilon$ -FOLLOW(S')={ $\$,e$ }使得 $S' \rightarrow \epsilon$ 也在 $M[S',e]$ 中
 $E \rightarrow b$

非终结符号	输入符号					
	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				



文法和自顶向下分析



- 设计文法(保证描述能力, 且适用于自顶向下分析)
 - 消除二义性
 - 消除左递归
 - 提取左公因子
- 自顶向下分析
 - 递归下降分析 (通用的可能需要回溯)
 - 预测分析
- LL(1)文法及其预测分析



第三题



题目 2 (LL 语法分析 (20 = 8 + 8 + 4 分))

考虑文法 G :

$$S \rightarrow cS \quad (1)$$

$$S \rightarrow AB \quad (2)$$

$$A \rightarrow aA \quad (3)$$

$$A \rightarrow \epsilon \quad (4)$$

$$B \rightarrow bBa \quad (5)$$

$$B \rightarrow d \quad (6)$$

(1) 请为文法 G 计算必要的 FIRST 集合与 FOLLOW 集合;

(2) 请为文法 G 构造预测分析表;

(3) 请问文法 G 是否是 $LL(1)$ 文法, 并说明理由。



第三题



- (1) FIRST集合与FOLLOW集合见下图
- (2) 预测分析表见下图
- (3) 因为预测分析表无冲突, G 是 $LL(1)$ 文法。

JFLAP : (ex8.5llparse-c.jff)

File Input Test Convert Help

Editor Build LL(1) Parse

Do Selected Do Step Do All Next Parse

Table Text Size

S	→ cS
S	→ AB
A	→ aA
A	→ λ
B	→ bBa
B	→ d

	FIRST	FOLLOW
A	{ λ, a }	{ b, d }
B	{ b, d }	{ a, \$ }
S	{ a, b, c, d }	{ \$ }

	a	b	c	d	\$
A	aA	λ		λ	
B		bBa		d	
S	AB	AB	cS	AB	

Table Text Size



第四题



题目 4 (语法制导定义与翻译 (20 分))

考虑如下文法 G ,

$$S \rightarrow L.L \mid L$$

$$L \rightarrow LB \mid B$$

$$B \rightarrow 0 \mid 1$$

请设计语法制导的翻译方案, 计算 S 产生的二进制小数对应的十进制值。如, $101.101_2 = 5.625_{10}$ 。你需要自行定义合适的属性。



第四题



首先定义合适的属性:

- $v(B) : B$ 对应的值
- $v(L) : L$ 对应的值
- $v(S) : S$ 对应的值
- $l(L) : L$ 的长度

以上均为综合属性。

然后给出语法制导的翻译方案:

$$S \rightarrow L_1.L_2 \quad \{v(S) = v(L_1) + v(L_2)/2^{l(L_2)}\}$$

$$S \rightarrow L \quad \{v(S) = v(L)\}$$

$$L_1 \rightarrow L_2B \quad \{v(L_1) = 2v(L_2) + v(B), \quad l(L_1) = l(L_2) + 1\}$$

$$L \rightarrow B \quad \{v(L) = v(B), \quad l(L) = 1\}$$

$$B \rightarrow 0 \quad \{v(B) = 0\}$$

$$B \rightarrow 1 \quad \{v(B) = 1\}$$